Elective Topic: Embedded System Design

Context and Objectives

It has become more economically feasible to process signals in a digital system. Analog signal processing cannot be changed in software, so it trades cost and space away in most applications. Digital signal processing typically happens in software on a PC, in firmware on a microcontroller, or on an FPGA.

Microcontrollers sit in the middle. They are an inexpensive way to process signals in hardware while staying flexible and programmable without weeks of lead time. They are usually the core of an embedded system. Devices inside an embedded system talk to each other over protocols such as SPI, I2C, 8080 and I2S, and each protocol exists for a reason.

Real-time audio is one of the least forgiving uses of a small microcontroller. The processor has to accept a new sample, do useful work on it, and produce an output sample before the next sample arrives. Miss that deadline once and the result is audible. The sampling instant also has to be exact. Jitter in the sample clock is indistinguishable from noise added to the signal.

For this design challenge you are to build an audio processing system on the ARM Peripheral Explorer (APE) board. The system takes audio in, processes it on the microcontroller, and plays it out through the speaker on the board. A guide and documentation will be provided to help you work with the APE board.

The sampling clock is not generated by the microcontroller. You will build it yourself out of discrete hardware. The microcontroller selects which of four sample rates that hardware produces, over SPI.

Your **primary objective** is to sample the audio input from an interrupt service routine triggered by your external timer, with the timer frequency set over SPI from the microcontroller.

Your **secondary objective** is to write up an LCD touch screen user interface for the user to control the audio interface device. Such a user interface should have application buttons, and page switching which leads to other extension tasks within this topic.

Suggested prerequisites: ELEC2141, DESN2000 / ELEC2142, ELEC3104
Courses that might help: COMP2521, ELEC4601, ELEC4621, ELEC4622



Figure 1. APE board

Task Distribution

This elective topic consists of one core objective, two extension tasks split into four parts, and one bonus task. Groups are 2 to 4 people.

Team size	Task load
2 people	Core objective, 1 extension part of choice, optional bonus task
3 people	Core objective, 2 extension parts of choice, optional bonus task
4 people	Core objective, 3 extension parts of choice, optional bonus task

Extension A and Extension B are each split into two parts. A.1 must be working before A.2 is attempted, and B.1 before B.2. Extensions A and B are independent of each other, so a group stuck on one can move to the other.

To obtain full marks the group must complete the core objective and its chosen extension parts. The bonus task is optional and is worth bonus marks. Completing more than the required task load does not grant extra marks. Leave the bonus task at the lowest priority. It has a low mark to effort ratio compared to the core objective.

Delegating tasks between group members matters, but we strongly recommend that you collaborate and cross help on each task.

Core Objective

The first step in audio processing is sampling the audio itself. Sampling by polling or from the main loop is not reliable. An interrupt elsewhere in the system will distort the sampling instant, and polling puts avoidable load on the processor.

In this task, you will build a hardware timer that triggers an interrupt service routine in the microcontroller, and sample the audio in that routine. Your timer must provide four frequencies: a base frequency f_x, and the three scaled frequencies $f_x/2$, $f_x/4$ and $f_x/8$.

You must also design the hardware so that it is SPI compatible with the microcontroller on the APE board, the STM32F103ZCT6. To change the output frequency, your board receives an SPI command from the APE board and outputs the frequency that command asks for.

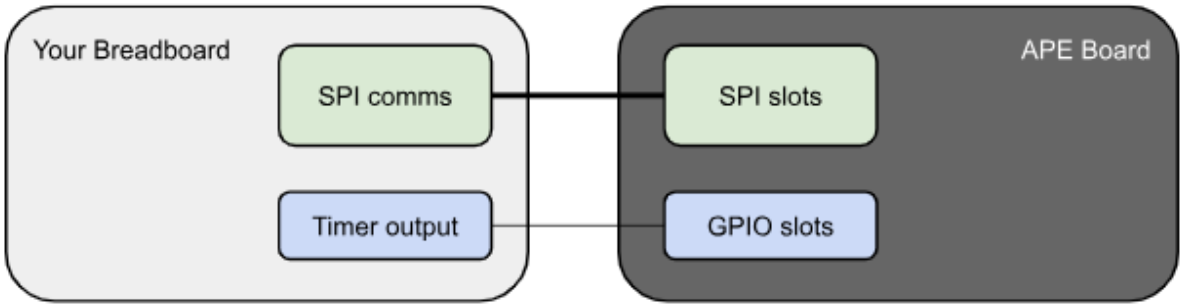


Figure 2. System diagram of objective

The Sampling Clock Generator

The base frequency for your group is:

f_x = (352 + (average of the last two digits of the zIDs of all team members)) kHz

Operating conditions:

1. The clock must be generated from hardware.
2. No clock or timing signal may be taken from the APE board and used to generate or discipline the sampling clock.
3. The sampling clock must not be taken from a laboratory signal generator. You may use a signal generator to test and characterise your circuit, but it cannot form part of your delivered design.
4. You may measure the sample rate using the microcontroller's own timers. That is not the same as deriving the sample timing from them.
5. Power supply: 3.3 V from APE Board output.

Basic requirements:

1. Four selectable output rates, at f_x, $f_x/2$, $f_x/4$ and $f_x/8$.
2. Frequency accuracy of each rate: ± 50%
3. Exactly one rate is presented to the microcontroller at any time.
4. The selected rate is held indefinitely until a valid command changes it.
5. The timer can be enabled and disabled on command and holds its selected rate across a disable and enable.
6. The output is a logic level clock suitable for direct connection to the APE board.

The SPI Command Set

The microcontroller sets the sample rate by sending a command to your timer hardware over SPI. The command set is fixed and is given below.

SPI command	Functionality
0xD1	Timer outputs at base frequency
0xD2	Timer outputs at base frequency / 2
0xD4	Timer outputs at base frequency / 4
0xD8	Timer outputs at base frequency / 8
0xE0	Timer disables
0xE1	Timer enables
0xFX	Timer outputs the closest available frequency to X kHz (bonus task)

Operating conditions:

1. The link is one directional. The microcontroller may drive data, clock, and any strobe or latch line that your design requires.
2. No signal needs to be routed from your timer hardware back into the APE board.

Basic requirements:

1. Every command in the table above is implemented, except 0xFX, which is the bonus task.
2. An invalid command must not affect the system. The selected rate must not change, reset, or glitch at any point during or after the reception and latching of an invalid command, and the enabled or disabled state must not change either.
3. A valid command changes the rate cleanly.

The Audio Path

Operating conditions:

1. Audio enters through the audio jack on the APE board and leaves through the speaker on the APE board.
2. The microcontroller core runs at 72 MHz from its own clock sources. Its core clock is not related to the sampling clock.
3. Sample timing must not be derived from any timer internal to the microcontroller.

Basic requirements:

1. Sampling happens in an interrupt service routine triggered by your external timer. Sampling by polling, or from the main loop, does not meet this requirement.
2. The output sample rate equals the input sample rate.
3. On the main page of the interface the system outputs the sampled audio with no processing. Audio going into the jack comes straight out of the speaker.
4. The system operates continuously at all four selectable rates, with no dropped samples and no repeated samples. (You need to convince your marker of this).
5. The output reproduces the input signal.
6. Aliasing at the input and reconstruction artefacts at the output must be addressed in your design. You must be able to measure the effect of what you did.
7. The system recovers to normal operation after a rate change without requiring a reset.

The User Interface

All runtime control of the system is through the LCD touchscreen on the APE board.

Basic requirements:

1. Page switching through to each extension part your group has attempted.
2. The timer frequency is selectable from the interface, and the currently selected rate is displayed.
3. The timer can be enabled and disabled from the interface.
4. The interface remains responsive while audio is running. Audio must not drop out while the screen is being redrawn or touched.

A minimal viable interface is shown below. We hope you build something more innovative than this.

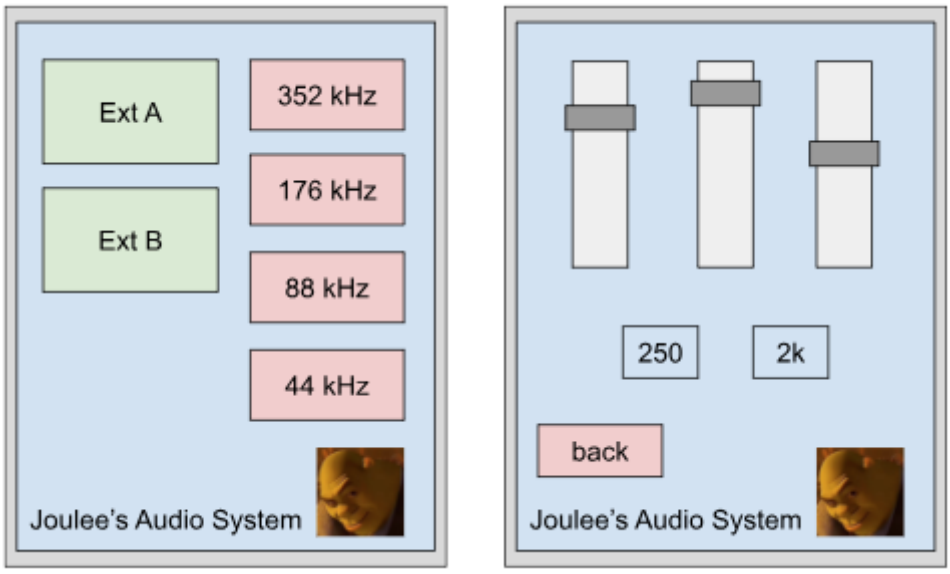


Figure 3. Minimal viable user interface for core and extension A

Tips

If a team member needs the breadboard circuit to get the other task working on the APE, they should put a really simple RC oscillator on the breadboard within the APE board to use while another team member is working on the SPI circuit.

Extension A: Audio Mixer (3 Band)

In music it is usually undesirable for one part of the spectrum to be much louder than the rest, and people differ on whether they want a bass heavy or treble heavy mix. An audio mixer lets the listener adjust the level of each part of the spectrum.

In this task you will split the audio signal into three frequency bands and build an interface to adjust the gain of each band.

MATLAB or other software can be used to obtain your filter coefficients, but you must understand the method you used to design the filter.

Operating conditions:

1. The core objective audio path continues to meet its requirements with the mixer running.
2. All gain and cutoff control is through the touchscreen.
3. Demonstrated acoustically through the speaker, and measured electrically at the DAC output.
4. Maximum input frequency is limited to 5kHz for this section.

A.1 requirements:

1. One band, running in real time at fx/8.
2. Gain adjustable from 0 to 2 from the interface.
3. The measured frequency response matches the response requested through the interface.
4. No dropped samples. (You must find a method to demonstrate this to the demonstrators).

A.2 requirements:

1. Three bands, running in real time, are able to run at fx/8.
2. Cutoff frequencies between bands: 500 Hz and 4 kHz.
3. Gain of each band adjustable from 0 to 2, independently.
4. The tone colour of the signal must be preserved.
5. The measured frequency response matches the response requested through the interface.
6. No dropped samples.

Both parts:

1. You should be able to explain the method you use and why you chose it, and what it costs you in computation. You must be able to show how close your design runs to the limit of the processor.

Soft requirements:

1. Configurable cutoff frequencies, within 50 percent either side of the values above.

Notes:

1. The speaker on board does not play low frequency sound well.

Tips

Set the optimization level to -O3 for the compiler (default -O0).

Extension B: Frequency Analysis

Displaying the raw signal in the time domain says very little. Frequency components shift in phase against each other faster than the eye can follow. Pitch, on the other hand, is something people can hear, and displaying it is a neat thing to do.

Your objective in this task is to display the frequency spectrum of the input in real time using a FFT algorithm.

Operating conditions:

1. The core objective audio path continues to meet its requirements while the display is running.
2. Both parts run at all four sample rates.
3. The displayed frequency range is 0 to 20 kHz.

B.1 requirements:

1. 128-point FFT, 128 bins.
2. The display refreshes at least 30 times a second, verified with a GPIO toggle on an oscilloscope.
3. No dropped samples while the display is running.

B.2 requirements:

1. The fundamental frequency of the input is estimated and displayed as a number.
2. Accuracy ±1 %, over 100 Hz to 2 kHz, for sine, square and triangle inputs.
3. The estimate updates at least 5 times a second.

Soft requirements:

1. A display of the power spectrum, with 8 bins.
2. A display of the time domain signal that holds position. A steady input must not drift along the time axis.

Tips

This task is purely algorithmic - in case the APE boards are fully occupied within the team, one can write programs in C or MATLAB to test the algorithm.

Bonus Task: Frequency Request

This task is optional and is worth bonus marks. Do not start it until your core objective and your chosen extension tasks are finished.

Implement the 0xFX command. The microcontroller sends a target frequency in kHz, and your hardware outputs the closest available frequency to it.

Basic requirements:

1. The hardware selects the available rate closest in frequency to the requested target. This is the nearest rate, not the highest rate below the target.
2. The selection is made in hardware. The microcontroller must not compute which rate to use, and must not send a rate index.
3. The link remains one directional. No signal may be routed from your hardware back into the APE board.
4. The behaviour on an invalid command continues to apply.

Components Available

1. The ARM Peripheral Explorer board, carrying an STM32F103ZCT6, an LCD touchscreen, an audio jack and a speaker. A guide and documentation will be provided.
2. 74HC595, 74HC153, 74HC93, 74HC74, 74HC02, 74HC00, 74HC14
3. Any components you were allowed to use in the Electronics topic.

You may not use any microcontroller, programmable logic device, or microprocessor other than the one on the APE board.

You may use laboratory test equipment to develop and characterise your design. Test equipment cannot form part of the delivered system.

Deliverables and Demonstration

You will demonstrate your system live in the laboratory. Have a test plan ready so that you know what you are showing and in what order.

At the demonstration you must be able to:

1. Show audio passing through the system at all four sample rates, with the rate selected from the touchscreen.
2. Show the measured frequency of your timer at each of the four settings.
3. Send an invalid command, and show that the selected rate and the enabled state do not change or glitch.
4. Demonstrate each extension part your group has attempted, against the requirements stated for it.

You must also have prepared:

1. A schematic of the timer hardware and all control logic, neatly drawn.
2. A description of your decode, including which byte patterns you treat as invalid.
3. Measured results supporting each requirement you claim to have met.
4. A journal containing working, measurements, design decisions, and dated progress.

Important Notes

Firmware Restrictions:

1. All firmware tasks must be done on Keil uVision 4 or uVision 5. A workaround for mac and Linux users will be provided.
2. No HAL functions or external libraries shall be used other than the ones provided.
3. AI can be used to assist you but shall not be used to generate code. Students are expected to understand every line of code they have submitted. Failure to explain any line of code can result in heavy penalization.

Hardware Precautions:

1. Never short the VCC and GND on the APE board. The fuse will break.
2. Never short the fuse slot. If the fuse is blown, please ask the demos for a new fuse.
3. Do not disassemble the hardware.
4. Do not drive a low impedance load without a driver or an amplifier.
5. Do not use a power supply with the board; all power sources within the project should come from the USB C power supply.

Supporting documents will be provided to assist you with your project.